

FraudSight

A Flagship Project Guide — Data Science Expert / AI Content Specialist

Contents

FraudSight	1
A Flagship Project Guide to Qualify for the Data Science Expert — AI Content Specialist Role	1
Competency coverage map	2
Phase 0 — Environment and reproducibility	3
Phase 1 — Framing, SQL feature engineering, and the leakage audit	3
Phase 2 — Baseline, and choosing the right metric under imbalance	4
Phase 3 — Cross-validation done correctly (nested + temporal)	5
Phase 4 — Unsupervised learning & dimensionality reduction	6
Phase 5 — Hyperparameter optimization (Bayesian)	6
Phase 6 — Deep learning in BOTH frameworks	7
Phase 7 — Statistical inference (Python + R) and Bayesian inference	8
Phase 8 — Light big-data with PySpark	9
Phase 9 — MLOps and CI/CD	10
Phase 10 — Capstone: the AI-code-audit portfolio	10
The failure-mode checklist (memorize this)	11
Suggested timeline	11

FraudSight

A Flagship Project Guide to Qualify for the Data Science Expert — AI Content Specialist Role

How to use this guide

This is a single, end-to-end project engineered so that finishing it forces you to demonstrate every competency the role screens for: rigorous problem design, ground-truth solutions in Python/R/SQL, deep learning in **both** PyTorch and TensorFlow, statistical inference, light big-data (Spark), and — most importantly — the ability to **audit AI-generated code** and catch the exact failure modes the job description names: data leakage, overfitting, and mishandled class imbalance.

Work through it in order. Each phase has three parts:

1. **Build** — what you implement.
2. **Prove** — the artifact that shows you can do it (a notebook cell, a plot, a metric, a written argument).

3. **Audit** — a deliberately flawed “AI-generated” solution you must critique and fix. This mirrors the day-to-day work of the role and is the part most candidates skip.

Do not skip the Audit sections. The role is not “build a model” — it is “judge whether someone else’s model reasoning is correct and explain why.” That skill is trained by the Audit blocks.

By the end you will have: one deployable model, a written **evaluation report**, a set of **ground-truth solution write-ups**, and a **red-team audit log** — a portfolio that maps one-to-one onto the four job duties (design challenges, author solutions, audit AI code, refine AI reasoning).

The dataset

Use a public transaction dataset with genuine class imbalance and a time dimension. Recommended: the **Kaggle Credit Card Fraud** dataset (284,807 transactions, 492 frauds $\approx$ 0.172% positives, PCA-anonymized features V1–V28 plus Time and Amount). If you want a raw, un-anonymized alternative to also practice SQL feature engineering, use the **IEEE-CIS Fraud Detection** dataset.

The extreme imbalance and temporal ordering are not incidental — they are the whole point. They are what make every downstream methodological choice (validation split, resampling, metric, threshold) consequential.

Competency coverage map

Job requirement	Where you prove it
Supervised learning	Phases 3, 5, 6
Unsupervised learning	Phase 4 (dimensionality reduction, anomaly detection)
Cross-validation strategies	Phase 3 (nested + temporal CV)
Hyperparameter optimization	Phase 5 (Bayesian optimization)
Bayesian inference	Phase 7 (posterior over metrics, Bayesian logistic regression)
Dimensionality reduction	Phase 4 (PCA, UMAP, autoencoder)
Class imbalance	Phase 2 & 6 (resampling, cost-sensitive learning, thresholding)
Deep learning — PyTorch	Phase 6a
Deep learning — TensorFlow/Keras	Phase 6b
Statistical inference	Phase 7 (bootstrap CIs, hypothesis tests, calibration)
Big data (Spark/Hadoop)	Phase 8 (PySpark feature pipeline)
Python / R / SQL	SQL in Phase 1, R in Phase 7, Python throughout
Scikit-Learn / PyTorch / TensorFlow	Phases 3–6
Data leakage detection	Phase 1 & every Audit block
Overfitting detection	Phase 3, 5, 6 Audits
MLOps / CI-CD	Phase 9
Auditing AI code / refining reasoning	Every Audit block + Phase 10 capstone

Phase 0 — Environment and reproducibility

Build. Create a reproducible environment. Pin versions; seed everything. A study that cannot be reproduced cannot be audited, and “is this reproducible?” is the first question you will ask of any AI-generated submission.

```
python -m venv .venv && source .venv/bin/activate
pip install "scikit-learn>=1.4" "pandas>=2.2" numpy scipy \
            "torch>=2.2" "tensorflow>=2.16" \
            optuna imbalanced-learn umap-learn shap \
            mlflow pyspark matplotlib jupyter
```

Set a single seed helper and use it everywhere:

```
import os, random, numpy as np
def seed_everything(seed=42):
    random.seed(seed); np.random.seed(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
    try:
        import torch; torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        torch.backends.cudnn.deterministic = True
    except ImportError: pass
    try:
        import tensorflow as tf; tf.random.set_seed(seed)
    except ImportError: pass
```

Prove. A requirements.txt with pinned versions and a notebook that runs top-to-bottom without manual intervention.

Audit. Take any public fraud-detection notebook (there are hundreds on Kaggle). List every source of non-determinism it ignores: unseeded resamplers, train_test_split without random_state, GPU non-determinism, shuffled data loaders. Write a one-paragraph reproducibility verdict. This is your first ground-truth critique.

Phase 1 — Framing, SQL feature engineering, and the leakage audit

Build. Before modeling, define the prediction target precisely. In fraud, the single most common fatal error is **target leakage** — using information that would not be available at scoring time. Examples: a is_chargeback flag posted days later, an account-level fraud rate computed over the full history including future transactions, or a Time-derived feature that encodes the label ordering.

Practice SQL feature engineering by loading the raw table into SQLite/DuckDB and building **point-in-time-correct** aggregates. The key discipline: every rolling feature must use only rows strictly before the current transaction.

```
-- Correct: rolling 24h spend PER CARD, strictly prior transactions only
SELECT
    t.txn_id,
    t.card_id,
    t.amount,
    COALESCE(SUM(p.amount), 0) AS spend_prior_24h,
    COALESCE(COUNT(p.txn_id), 0) AS txn_count_prior_24h
FROM txns t
```

```
LEFT JOIN txns p
  ON p.card_id = t.card_id
  AND p.ts < t.ts -- strictly before: no leakage
  AND p.ts >= t.ts - INTERVAL '24 hours'
GROUP BY t.txn_id, t.card_id, t.amount;
```

The `p.ts < t.ts` predicate is the whole game. Using `<=` leaks the current row; joining without the time bound leaks the future.

Prove. A documented feature dictionary: for each feature, its formula, its availability time, and a one-line leakage justification.

Audit — the leakage trap. Here is an “AI-generated” feature snippet. Find the bug before reading the answer.

```
# AI-generated feature engineering (contains a leak)
df["card_fraud_rate"] = df.groupby("card_id")["is_fraud"].transform("mean")
df["amount_z"] = (df["amount"] - df["amount"].mean()) / df["amount"].std()
X_train, X_test = train_test_split(df, test_size=0.2, random_state=42)
```

Ground-truth critique. Two leaks. (1) `card_fraud_rate` is computed over the entire dataframe including test rows and including each row’s own label — classic target leakage; it must be computed with a time-aware expanding mean excluding the current row, and fit only on training data. (2) `amount_z` uses global mean/std computed before the split, so test statistics leak into training scaling; standardization parameters must be fit on train and applied to test (use a Pipeline). Both inflate offline metrics and collapse in production. Rewrite: compute all statistics inside a sklearn Pipeline/ColumnTransformer fit only on training folds.

Phase 2 — Baseline, and choosing the right metric under imbalance

Build. Fit a logistic-regression baseline. Then confront the imbalance head-on: with 0.172% positives, **accuracy is useless** (a model predicting “never fraud” scores 99.83%). Establish the metrics you will actually use:

- **PR-AUC (average precision)** — the primary metric under heavy imbalance; it focuses on the positive class and is not inflated by the huge true-negative mass the way ROC-AUC is.
- **ROC-AUC** — report it, but understand its optimism under imbalance.
- **Precision@k / Recall@k** — operationally, a fraud team can only review so many flagged transactions per day; report precision within the top-k budget.
- **Cost-weighted metric** — assign a dollar cost to false negatives (missed fraud) vs false positives (blocked legit customer) and optimize expected cost.

Prove. A metrics table for the baseline, plus a written paragraph justifying PR-AUC as primary. Include the “always-negative” model in the table to make the accuracy fallacy concrete.

Audit — the metric trap.

```
# AI-generated evaluation (misleading under imbalance)
from sklearn.metrics import accuracy_score
print("Accuracy:", accuracy_score(y_test, model.predict(X_test))) # 0.9992 -
  ↳ "great!"
```

Ground-truth critique. Accuracy of 0.9992 is *worse* than the trivial always-negative baseline once you account for the base rate, and it says nothing about caught fraud. Replace with `average_precision_score` (PR-AUC), `roc_auc_score`, and a confusion matrix at an operating threshold chosen from the business cost matrix — never the default 0.5.

Phase 3 — Cross-validation done correctly (nested + temporal)

Build. This phase separates competent practitioners from the rest. Implement two things:

1. **Stratified K-fold** for a first pass, so each fold preserves the tiny positive rate. Random K-fold can hand a fold zero positives.
2. **Temporal / walk-forward validation.** Transactions are time-ordered; a random split lets the model train on the future and test on the past — leakage. Use `TimeSeriesSplit` or an explicit expanding window so training always precedes validation in time.
3. **Nested CV** for honest performance estimates when you also tune hyperparameters (Phase 5): an outer loop for performance estimation, an inner loop for tuning, so the tuning never sees the outer test fold.

```
from sklearn.model_selection import TimeSeriesSplit, StratifiedKFold,
    ↪ cross_val_score

# Temporal validation — train always precedes validation
tscv = TimeSeriesSplit(n_splits=5)

# Nested CV skeleton
from sklearn.model_selection import GridSearchCV
outer = StratifiedKFold(5, shuffle=True, random_state=42)
inner = StratifiedKFold(3, shuffle=True, random_state=42)
# search = GridSearchCV(estimator, param_grid, cv=inner,
    ↪ scoring="average_precision")
# nested_scores = cross_val_score(search, X, y, cv=outer,
    ↪ scoring="average_precision")
```

Prove. A comparison: PR-AUC under random K-fold vs temporal CV. The gap (usually random CV is optimistically higher) is your evidence that you understand why validation design matters.

Audit — the CV trap.

```
# AI-generated CV (two structural errors)
from imblearn.over_sampling import SMOTE
X_res, y_res = SMOTE().fit_resample(X, y)          # (1)
scores = cross_val_score(clf, X_res, y_res, cv=5)  # (2)
```

Ground-truth critique. (1) SMOTE is applied to the **entire dataset before** cross-validation, so synthetic points derived from validation-fold neighbors leak across folds — resampling must happen **inside** each training fold via an `imblearn.Pipeline`. (2) `cv=5` is plain `KFold` on time-ordered data; it lets the model see the future. Correct version:

```
from imblearn.pipeline import Pipeline
pipe = Pipeline([("smote", SMOTE()), ("clf", clf)])
scores = cross_val_score(pipe, X, y, cv=TimeSeriesSplit(5),
    scoring="average_precision")
```

Now resampling is refit per fold and validation always follows training in time.

Phase 4 — Unsupervised learning & dimensionality reduction

Build. Demonstrate the unsupervised half of the job three ways:

1. **PCA** — linear reduction; inspect explained-variance ratio and reconstruct. Note that V1–V28 are *already* PCA components in the Kaggle set, so run your PCA on the raw IEEE-CIS features or on engineered features to make it meaningful.
2. **UMAP** (or t-SNE) — non-linear 2-D embedding to visually separate fraud clusters. Discuss why t-SNE/UMAP distances are not globally metric and must not be fed downstream as if they were.
3. **Autoencoder anomaly detection** — train an autoencoder on legitimate transactions only; flag high reconstruction-error transactions as anomalies. This is unsupervised fraud detection and doubles as a deep-learning warm-up.

Prove. Explained-variance curve, a 2-D UMAP scatter colored by label, and a reconstruction-error histogram separating classes. Report PR-AUC of the reconstruction-error score as a standalone detector.

Audit — the reduction trap.

```
# AI-generated: PCA before the split, and used as a preprocessing "cleanup"
pca = PCA(n_components=10).fit(X)      # fit on all data → leakage
X_pca = pca.transform(X)
X_train, X_test = train_test_split(X_pca, ...)
```

Ground-truth critique. PCA is fit on the full dataset including test rows — the principal axes are informed by test data, a subtle leak. Fit PCA inside a pipeline on training folds only. Second, reducing to 10 components “to clean noise” is an unjustified assumption; validate `n_components` by downstream PR-AUC, not by eyeballing variance. t-SNE/UMAP embeddings should **never** be used as model features — they are non-parametric, non-reproducible across runs, and have no transform for new points (t-SNE) or an unstable one (UMAP).

Phase 5 — Hyperparameter optimization (Bayesian)

Build. Move beyond grid search. Use **Bayesian optimization** (Optuna’s TPE sampler) to tune a gradient-boosted model (XGBoost/LightGBM) against PR-AUC under temporal CV. Explain *why* Bayesian optimization beats grid/random search: it models the objective as a surrogate (e.g., a Gaussian process or tree-structured Parzen estimator) and spends evaluations where expected improvement is highest.

```
import optuna
from sklearn.model_selection import cross_val_score, TimeSeriesSplit

def objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 200, 1200),
        "max_depth": trial.suggest_int("max_depth", 3, 10),
        "learning_rate": trial.suggest_float("learning_rate", 1e-3, 0.3, log=True),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "scale_pos_weight": trial.suggest_float("scale_pos_weight", 1, 600,
        ↪ log=True),
    }
```

```

model = LGBMClassifier(**params, random_state=42)
return cross_val_score(model, X, y, cv=TimeSeriesSplit(5),
                        scoring="average_precision").mean()

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=60)

```

Prove. An optimization-history plot, the best trial's params, and a comparison of Bayesian vs random search at equal trial budget. Discuss `scale_pos_weight` as the imbalance lever inside the search.

Audit — the tuning/overfitting trap.

```

# AI-generated tuning (overfits the validation set)
best = None
for lr in np.linspace(0.001, 0.5, 200):
    model.set_params(learning_rate=lr).fit(X_train, y_train)
    score = average_precision_score(y_val, model.predict_proba(X_val)[: ,1])
    best = max(best, score) # picks the lr that maxes the SAME val set
print("Best val PR-AUC:", best)

```

Ground-truth critique. Selecting over 200 configurations on one fixed validation set **overfits the validation set** — the reported best score is an optimistic, upward-biased estimate. With nested CV the tuning happens in the inner loop and the outer loop gives an unbiased estimate; report the outer-loop score, never the inner best. Also, tuning only `learning_rate` while holding `n_estimators` fixed is confounded (they trade off); tune them jointly with early stopping.

Phase 6 — Deep learning in BOTH frameworks

You must be able to read, write, and **audit** code in both PyTorch and TensorFlow, because AI-generated submissions arrive in either. Build the *same* tabular fraud classifier twice.

Phase 6a — PyTorch

```

import torch, torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

class FraudMLP(nn.Module):
    def __init__(self, d_in):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_in, 128), nn.ReLU(), nn.BatchNorm1d(128), nn.Dropout(0.3),
            nn.Linear(128, 64), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(64, 1)) # logits; no sigmoid here
    def forward(self, x): return self.net(x).squeeze(1)

# Handle imbalance with a weighted loss, NOT by oversampling the val set
pos_weight = torch.tensor([(len(y_tr) - y_tr.sum()) / y_tr.sum()])
loss_fn = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

```

Key teaching points to write up: `BCEWithLogitsLoss` (numerically stable, fold sigmoid into

the loss), `pos_weight` for imbalance, `model.eval()/torch.no_grad()` at inference, and never calling `BatchNorm1d` with batch size 1.

Phase 6b — TensorFlow / Keras

```
import tensorflow as tf
model = tf.keras.Sequential([
    tf.keras.layers.Input((d_in,)),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=[tf.keras.metrics.AUC(curve="PR", name="pr_auc")])
model.fit(X_tr, y_tr, validation_data=(X_val, y_val),
        class_weight={0: 1.0, 1: pos_weight_value},
        epochs=50, callbacks=[tf.keras.callbacks.EarlyStopping(
            monitor="val_pr_auc", mode="max", patience=5,
            ↪ restore_best_weights=True)])
```

Prove. Both models reach comparable PR-AUC; produce a side-by-side table and a short essay on framework-equivalent concepts (eager vs graph, `DataLoader` vs `tf.data`, manual loop vs `.fit`).

Audit — the deep-learning trap.

```
# AI-generated Keras training (three bugs)
model.add(Dense(1, activation="sigmoid"))
model.compile(loss="binary_crossentropy", metrics=["accuracy"]) # (1)
model.fit(X, y, epochs=200) # (2)
scaler = StandardScaler().fit(X); X = scaler.transform(X) # (3)
```

Ground-truth critique. (1) accuracy metric under 0.17% positives is meaningless — use PR-AUC. (2) 200 epochs with no validation split, no early stopping → overfitting with no way to detect it; add a temporal validation split and early stopping on `val_pr_auc`. (3) The scaler is fit on all of `X` including what will become validation/test → leakage; fit on train only. Also no imbalance handling (`class_weight` missing), so the model may collapse to the majority class.

Phase 7 — Statistical inference (Python + R) and Bayesian inference

Build. The role explicitly requires statistical inference and Bayesian inference, and precision about statistical conclusions. Demonstrate:

1. **Bootstrap confidence intervals** for PR-AUC — a single point estimate is not a conclusion. Resample the test set 2,000× and report a 95% CI.
2. **Model comparison with a hypothesis test** — is model A's PR-AUC genuinely higher than model B's, or noise? Use a paired bootstrap or a DeLong test for ROC-AUC; state the null hypothesis, the test statistic, and interpret the p-value correctly (it is *not* the probability the null is true).

3. **Calibration** — a fraud “probability” of 0.9 should mean ~90% of such transactions are fraud. Plot a reliability curve; apply Platt scaling or isotonic regression; report Brier score and Expected Calibration Error.
4. **Bayesian logistic regression** — fit the baseline in a Bayesian framework (PyMC or brms in R) to get **posterior distributions** over coefficients and over the fraud probability, plus credible intervals. Contrast a 95% *credible* interval (“95% probability the parameter lies here, given priors and data”) with a 95% *confidence* interval (a statement about the procedure’s long-run coverage). Getting this distinction right in writing is exactly the kind of precision the role audits for.

Do the calibration/CI work in **R** as well to prove R fluency:

```
# R: bootstrap 95% CI for average precision, and a reliability check
library(boot)
ap <- function(d, i) PRROC::pr.curve(scores.class0 = d$score[i][d$y[i]==1],
                                     scores.class1 =
                                     ↪ d$score[i][d$y[i]==0])$auc.integral
b <- boot(data.frame(score=scores, y=y_test), statistic=ap, R=2000)
boot.ci(b, type="perc")      # percentile 95% CI
```

Prove. A results section reporting PR-AUC $\pm$ 95% CI, a calibration plot before/after, and a posterior plot with credible intervals. Write one paragraph interpreting each — correctly.

Audit — the inference trap.

```
# AI-generated claim
# "Model A (PR-AUC 0.842) is better than Model B (0.838), a clear improvement."
```

Ground-truth critique. A 0.004 gap with no interval and no test is not a conclusion. Compute a paired bootstrap CI on the *difference*; if it straddles zero, the “clear improvement” claim is unsupported. Also flag if the two models were compared on different splits or if the gap is within run-to-run seed variance (report the seed-variance band).

Phase 8 — Light big-data with PySpark

Build. Show you can move the same logic to a distributed engine. Re-implement the Phase 1 point-in-time feature aggregation in **PySpark** using a window function, and train a Spark MLlib model. The teaching point is that the leakage-safe windowing must be preserved when you scale out.

```
from pyspark.sql import Window
from pyspark.sql import functions as F

w = (Window.partitionBy("card_id").orderBy("ts")
     .rangeBetween(-86400, -1))      # prior 24h, EXCLUDING current row
df = df.withColumn("spend_prior_24h", F.sum("amount").over(w))
```

Note `rangeBetween(-86400, -1)` ends at -1, excluding the current row — the Spark equivalent of the `p.ts < t.ts` SQL predicate. Getting the boundary wrong is the distributed version of the Phase 1 leak.

Prove. A PySpark notebook producing identical features to the SQL version on a sample (assert equality), plus an MLlib GBTCClassifier PR-AUC. One paragraph on when distributed compute is and is not worth it (most fraud datasets fit in memory; Spark earns its keep at billions of rows).

Audit — the big-data trap. Given an AI-generated Spark snippet using `rangeBetween(-86400, 0)` (includes current row) or `rowsBetween` instead of `rangeBetween` (counts rows, not seconds), identify the leak / semantic error and correct it.

Phase 9 — MLOps and CI/CD

Build. Turn the model into a maintained system.

- **Experiment tracking** with MLflow: log params, metrics, artifacts, and the exact data hash for every run.
- **Model packaging:** a predict service (FastAPI) that loads the trained pipeline and returns a calibrated probability plus the operating-threshold decision.
- **CI:** a GitHub Actions workflow that runs unit tests (feature correctness, no-leakage assertions), a data-validation step, and a “training smoke test” on every push.
- **Monitoring plan (written):** define **data drift** (input distribution shift) and **concept drift** (fraud patterns evolve), the metrics you’d monitor (PSI on features, rolling PR-AUC on labeled feedback), and a retraining trigger.

```
# .github/workflows/ci.yml (excerpt)
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install -r requirements.txt
      - run: pytest tests/ -q           # includes leakage + calibration unit tests
```

Include a genuinely useful leakage unit test:

```
def test_no_future_leakage():
    # scoring row must not change if future rows are appended
    base = build_features(txns_up_to(t)).loc[txn_id]
    with_future = build_features(txns_up_to(t_plus)).loc[txn_id]
    pd.testing.assert_series_equal(base, with_future)
```

Prove. A repo with green CI, an MLflow screenshot, and a written monitoring/retraining runbook.

Audit — the MLOps trap. Critique an AI-generated pipeline that pickles a bare model (not the fitted preprocessing pipeline), so production applies raw, unscaled inputs — a silent train/serve skew. Explain train/serve skew and why the *entire* pipeline (preprocessing + model) must be the serialized artifact.

Phase 10 — Capstone: the AI-code-audit portfolio

This phase *is* the job. Assemble everything into the deliverables an AI Content Specialist actually produces.

1. **Design 5 original challenges.** For each competency cluster (imbalance handling, CV design, tuning, dimensionality reduction, Bayesian inference), write a self-contained

problem statement with a dataset, a clear task, and hidden pitfalls — the kind you’d give an AI model to solve.

2. **Author 5 ground-truth solutions.** For each challenge, write the definitive, expert solution: code that runs, math that’s correct, and prose explaining *why* each choice is right. These are your reference answers.
3. **Run a red-team audit log.** Take AI-generated solutions to your challenges (generate them with any LLM, or reuse the flawed snippets above), and for each produce a structured critique:
 - **Verdict** (correct / flawed / subtly flawed)
 - **Failure mode** (leakage / overfitting / imbalance mishandling / invalid statistical claim / other)
 - **Evidence** (the specific line and why it fails)
 - **Fix** (corrected code)
 - **Reasoning feedback** (how the model’s *thought process* went wrong — the “refine AI reasoning” duty)
4. **Write the evaluation report.** A 3–5 page document tying the model’s final performance to honest, interval-based statistical conclusions, calibration, and known limitations.

Prove. These four artifacts — challenges, solutions, audit log, report — are your portfolio. They map exactly onto the four job duties: *design challenges, author ground-truth solutions, audit AI-generated code, refine AI reasoning.*

The failure-mode checklist (memorize this)

Every audit in this guide is an instance of one of these. When you review any AI-generated data-science submission, run this list:

1. **Data leakage** — target leakage, train/test contamination, resampling before splitting, scaling/PCA fit on all data, non-point-in-time features, look-ahead in time series.
2. **Overfitting** — tuning on a fixed validation set, no early stopping, reporting inner-loop CV scores, too-flexible model on tiny positive class.
3. **Class-imbalance mishandling** — accuracy as the metric, default 0.5 threshold, resampling the validation/test set, no cost weighting.
4. **Invalid statistical conclusions** — point estimates with no CI, misreading p-values, confusing confidence vs credible intervals, ignoring seed variance, uncalibrated probabilities treated as probabilities.
5. **Reproducibility / engineering** — unseeded randomness, train/serve skew, serializing the model without its preprocessing, non-deterministic GPU ops.

If you can spot, explain, and fix all five families on sight, you are qualified for the role.

Suggested timeline

Weeks	Phases	Output
1	0–2	Reproducible env, leakage-safe features, metric framework
2	3–4	Nested/temporal CV, dimensionality reduction, anomaly detector
3	5–6	Bayesian HPO, PyTorch + TF models
4	7–8	Statistical/Bayesian inference (Python+R), PySpark
5	9	MLOps: CI, tracking, serving, monitoring runbook

Weeks	Phases	Output
6	10	Capstone portfolio: challenges, solutions, audit log, report

Six focused weeks. The output is a deployable model **and** a portfolio that demonstrates, artifact by artifact, every skill the role screens for — with the audit work that most candidates never do front and center.